

Modelagem de Software

A modelagem de software é uma etapa crucial no desenvolvimento de sistemas, permitindo que todos os envolvidos tenham uma visão clara e estruturada do projeto antes de iniciar a codificação. Ao contrário de partir diretamente para a implementação, a modelagem oferece uma abstração que facilita a compreensão das funcionalidades e requisitos do software. Isso é especialmente importante para sistemas complexos, onde a falta de clareza pode gerar retrabalho, atrasos e falhas na comunicação entre os stakeholders. Hoje, exploraremos o papel da modelagem, os cenários em que ela é indicada e quando pode ser dispensada, sempre destacando a importância de compreender o sistema como um todo.

Por que modelar sistemas?

A modelagem de sistemas é uma etapa essencial no desenvolvimento de software, pois, ao contrário de partir diretamente para a programação, a criação de modelos oferece uma visão clara sobre o que está sendo desenvolvido. Modelos permitem que todos os envolvidos no processo compreendam melhor as necessidades e a estrutura do sistema, facilitando a comunicação entre os diferentes stakeholders, como desenvolvedores, designers e clientes. Nem sempre a modelagem é necessária, e ao longo desta aula, exploraremos os cenários em que essa prática é recomendada ou dispensável.

Um exemplo prático para ilustrar a importância da modelagem pode ser encontrado na construção de um imóvel. Imagine que você está prestes a financiar a compra de um imóvel que ainda não foi construído. Para tomar essa decisão, será necessário consultar uma planta do imóvel. Essa planta, que é um modelo, oferece uma visão antecipada de como o imóvel será, ajudando você a tomar decisões baseadas em critérios importantes, como a localização dos quartos em relação ao elevador ou à garagem. Esse mesmo princípio se aplica à modelagem de software: assim como a planta do imóvel,

o modelo de software ajuda a visualizar como o sistema será antes de ser desenvolvido.

Modelos de software servem como uma abstração, auxiliando na compreensão de sistemas complexos. Eles podem ser tanto físicos quanto intangíveis, como, por exemplo, um modelo de carro em argila antes de sua fabricação ou um diagrama UML para o desenvolvimento de software. Esses modelos ajudam a planejar e a antecipar problemas que possam surgir durante a implementação, economizando tempo e recursos. Além disso, eles proporcionam uma comunicação visual clara, o que é muito mais eficiente do que descrições longas e técnicas, que podem ser difíceis de interpretar.

No contexto de desenvolvimento de software, os modelos ajudam os desenvolvedores a entenderem melhor o que estão construindo, permitindo uma visão mais clara sobre o design, a arquitetura e os requisitos do sistema. Antes de partir para a codificação, é essencial que todos compreendam os objetivos e as funcionalidades do software. A modelagem é uma ferramenta poderosa para facilitar essa compreensão, evitando retrabalho e falhas de comunicação.

Por fim, a modelagem não se aplica apenas ao desenvolvimento de software. Sistemas de transporte, processos corporais e até a gestão de registros imobiliários utilizam modelos para facilitar a visualização e o planejamento. Através da modelagem, podemos lidar com a complexidade de maneira mais eficiente, garantindo que o sistema final atenda às expectativas de todos os envolvidos. Portanto, entender por que e quando modelar é uma habilidade essencial para quem trabalha com desenvolvimento de sistemas.

Quando dispensar a modelagem

A modelagem de software é uma prática amplamente recomendada, mas não é necessária em todos os cenários. Agora, vamos explorar situações em que a modelagem pode ser dispensada e por que, em determinados casos, não é necessário adotar essa estratégia. Primeiramente, é importante destacar que, embora os modelos sejam úteis em muitos contextos, há situações onde partir diretamente para a implementação é mais eficiente e prático.

Um exemplo claro disso pode ser a construção de uma casinha de cachorro no quintal de casa. Será que realmente precisamos de um projeto elétrico, hidráulico ou estrutural para algo tão simples? Ou será que devemos criar uma maquete antes de começar a construir? Provavelmente, não. Da mesma forma, se estivermos criando uma macro no Microsoft Word ou uma fórmula avançada no Excel, não há necessidade de elaborar diagramas de modelagem complexos. Esses são exemplos práticos em que partir diretamente para a implementação é mais adequado do que investir tempo em modelagem.

Outro cenário em que a modelagem pode ser dispensada é no desenvolvimento de sistemas muito simples, como um aplicativo conversor de moedas. Se o objetivo é apenas converter de dólar para real ou de euro para dólar, o uso de modelos pode ser desnecessário. Nesses casos, o tempo gasto na criação de modelos formais pode não compensar, já que o sistema é simples e pode ser desenvolvido diretamente através de testes e ajustes rápidos.

Existem alguns critérios que podem ser usados para determinar quando a modelagem pode ser deixada de lado. O primeiro é se o domínio do problema é bem conhecido. Quando trabalhamos em áreas familiares, como sistemas de vendas ou bibliotecas, onde o funcionamento já é bem compreendido, o uso de modelos pode ser excessivo. Além disso, quando a solução é simples e fácil de construir, como no caso do conversor de moedas, o tempo investido na modelagem pode não ser justificável. Outro critério é quando poucas pessoas estão envolvidas no desenvolvimento ou uso do sistema, já que a comunicação direta pode ser suficiente sem a necessidade de modelos formais.

Outros fatores que indicam que a modelagem pode ser dispensada incluem a baixa necessidade de manutenção contínua e a pouca probabilidade de que o escopo do sistema cresça significativamente no futuro. Se o sistema não precisar de atualizações frequentes e não houver uma expectativa de grandes mudanças nas necessidades do cliente, o uso de modelos pode ser considerado um esforço desnecessário.

É importante, no entanto, evitar cair em armadilhas que levam ao não uso de modelos pelos motivos errados. Muitos desenvolvedores dispensam a modelagem porque possuem um profundo conhecimento da linguagem de programação, acreditam que a solução não evoluirá ou temem que o processo de modelagem atrase o desenvolvimento. Esses são erros comuns que podem comprometer a qualidade do sistema no longo prazo.

Motivação de uso de modelos

A motivação para o uso de modelos no desenvolvimento de software é um tema central nesta terceira parte da nossa aula. Muitos desenvolvedores, especialmente os mais novos, podem questionar se vale a pena investir tempo em modelagem, já que a vontade de partir diretamente para a codificação é grande. No entanto, esse impulso pode ser perigoso, pois ignorar a modelagem pode aumentar os riscos envolvidos no projeto, como atrasos no prazo de entrega, estouro do orçamento e falhas no escopo ou na qualidade do software final.

Uma das principais razões para usar modelos é a complexidade. Em projetos complexos, há muitos fatores a considerar, como a clareza na comunicação entre a equipe, a colaboração eficaz, o controle de prazos e o gerenciamento de custos. Imagine que um arquiteto decida construir um prédio de dez andares sem antes criar um projeto. Isso seria impensável, pois todos os envolvidos na obra dependem dos modelos para entender e colaborar no processo de construção. Da mesma forma, no desenvolvimento de software, pular a fase de modelagem pode resultar em problemas técnicos e econômicos.

Modelar também é crucial quando pensamos em sistemas críticos, como softwares usados em foguetes ou em dispositivos médicos que monitoram pacientes em hospitais. Não podemos simplesmente colocar esses sistemas em produção e esperar que tudo funcione corretamente. Em muitos casos, eles são difíceis ou até impossíveis de testar diretamente no ambiente real, e a modelagem permite simular diferentes cenários e avaliar os riscos antes da implementação.

Além de sistemas que afetam a vida humana, também existem sistemas críticos para negócios. Uma falha em um software financeiro, por exemplo, pode gerar enormes perdas para a empresa. A modelagem permite que todos os stakeholders visualizem o sistema de forma completa, assim como uma maquete permite que um arquiteto visualize um grande condomínio. Com os modelos, é possível tomar decisões mais acertadas no início do processo, evitando surpresas desagradáveis mais adiante.

Outro aspecto importante é a combinação de elementos visuais e textuais na modelagem. Muitas vezes, as representações textuais são difíceis de interpretar, enquanto os modelos visuais tornam a compreensão mais clara e acessível. Isso facilita a comunicação e a manutenção do software em todas as fases do seu ciclo de vida, desde a concepção até as atualizações futuras.

Na indústria automotiva, por exemplo, um carro moderno pode ter dezenas de componentes eletrônicos, todos controlados por software. Desde o sistema de alarme até o piloto automático, passando pelos sistemas de entretenimento e navegação, todos esses elementos dependem de modelos bem construídos para funcionarem de forma integrada e eficaz. Em resumo, a modelagem de software é essencial para gerenciar a complexidade e entender os riscos associados ao desenvolvimento de sistemas complexos, sejam eles críticos para a vida humana ou para o sucesso de uma organização.

Design e arquitetura de software

A arquitetura de software e o design de software são dois conceitos importantes e frequentemente confundidos, mas têm papéis distintos no desenvolvimento de sistemas. Para se tornar um arquiteto de software de excelência, é necessário dominar o design de software, especialmente o design orientado a objetos. Portanto, é fundamental entender como esses dois conceitos se diferenciam e se complementam.

A arquitetura de software é responsável pela estrutura geral de um sistema. Ela considera o sistema como um todo, abrangendo seus componentes, suas interações e seus relacionamentos. A arquitetura também envolve decisões importantes sobre frameworks, armazenamento de dados e como os

diferentes componentes do sistema vão interagir. O arquiteto de software é o responsável por garantir a integridade conceitual do sistema, ou seja, ele conhece profundamente o ambiente em que o sistema vai operar e garante que o conceito inicial do projeto seja mantido ao longo de todo o desenvolvimento. Além disso, ele atua como a ponte entre o cliente, os desenvolvedores e o produto final, garantindo que todos estejam alinhados com os objetivos do projeto.

Por outro lado, o design de software se concentra nos detalhes dos componentes individuais do sistema. Enquanto a arquitetura trata de uma visão mais ampla, o design de software se preocupa com a modularidade, a reutilização e a flexibilidade do código, permitindo que o sistema seja escalável e fácil de manter. O design envolve a criação de módulos independentes que podem ser reutilizados em diferentes partes do sistema, além de se utilizar de notações visuais, como os diagramas de classe UML, para representar a estrutura e o comportamento do software de maneira clara e organizada. Princípios como a separação de conceitos e a ocultação de informações também são fundamentais no design de software.

A diferença central entre arquitetura e design é o nível de abstração. A arquitetura lida com aspectos de alto nível, oferecendo uma visão geral do sistema, enquanto o design trata dos detalhes de baixo nível, focando em como cada parte do sistema será implementada. Apesar dessa distinção, em muitos casos, esses dois papéis se fundem na prática, especialmente em empresas menores ou em projetos menos complexos, onde o profissional atua tanto como arquiteto quanto como designer.

Exemplos práticos ajudam a ilustrar a importância desses papéis. Em um projeto pequeno, como o desenvolvimento de um aplicativo simples, talvez não seja necessário diferenciar tão claramente o arquiteto do designer. No entanto, em projetos complexos, como o desenvolvimento de um sistema de controle de tráfego aéreo ou de uma plataforma de e-commerce, a definição clara de arquitetura e design é essencial para garantir que o sistema seja escalável e possa evoluir ao longo do tempo sem se tornar caótico.

Por fim, tanto o arquiteto quanto o designer de software precisam ter habilidades técnicas e de comunicação, já que vão interagir com diversos

stakeholders ao longo do processo de desenvolvimento. Saber articular bem suas ideias e compreender as necessidades de todos os envolvidos é tão importante quanto dominar as técnicas de desenvolvimento.

IDEs e ferramentas CASE

Nesta quinta parte da nossa aula, abordaremos o tema de IDEs (Ambientes de Desenvolvimento Integrado) e ferramentas CASE (Computer-Aided Software Engineering). Esses dois tipos de ferramentas são essenciais para o desenvolvimento moderno de software, pois oferecem suporte tanto na criação quanto na manutenção de sistemas complexos, otimizando o tempo dos desenvolvedores e garantindo mais precisão no processo de desenvolvimento.

As IDEs, ou Ambientes de Desenvolvimento Integrado, são plataformas que reúnem todas as ferramentas necessárias para desenvolver software em um único ambiente. Elas incluem editores de código-fonte, compiladores, depuradores e outras ferramentas essenciais. Antes da existência das IDEs, os programadores utilizavam editores de texto simples para escrever o código e, em seguida, executavam o compilador manualmente. Caso houvesse erros, era necessário voltar ao editor, corrigir o problema e repetir o processo, o que tornava o desenvolvimento demorado e sujeito a falhas. Um exemplo marcante de IDE foi o Turbo Pascal, lançado pela Borland em 1991, que já integrava todas essas ferramentas em um único ambiente.

No contexto da nossa disciplina, trabalharemos com o Android Studio, a IDE oficial do Google para o desenvolvimento de aplicativos Android. Essa ferramenta permite a criação de aplicações para diversos dispositivos, como celulares, tablets, smartwatches e sistemas multimídia para carros. O Android Studio também possui um emulador, que permite testar os aplicativos em um ambiente simulado antes de instalá-los em dispositivos reais. Para instalar essa IDE, basta acessar o site oficial do Android Studio e seguir o processo de instalação, que detecta automaticamente o sistema operacional e oferece as opções adequadas.

Além das IDEs, também é importante conhecer as ferramentas UML (Unified Modeling Language). Essas ferramentas são usadas para criar diagramas que

representam graficamente a estrutura e o comportamento de um software. O uso de uma ferramenta específica para UML, como o **Asta UML** — que utilizaremos em nosso curso —, facilita o processo de criação de diagramas e a integração com o código-fonte. Essas ferramentas automatizam várias tarefas, como a geração de código a partir dos diagramas ou a criação de diagramas a partir do código já existente, tornando o processo de desenvolvimento mais eficiente.

As ferramentas CASE, por sua vez, têm como objetivo auxiliar nas atividades de engenharia de software, automatizando diversas tarefas, como a geração de código-fonte. Hoje em dia, com a popularização do conceito de low-code, essas ferramentas permitem que pessoas sem experiência em programação possam desenvolver aplicativos utilizando componentes visuais e técnicas de arrastar e soltar (drag-and-drop). Um exemplo de destaque é o GeneXus, uma ferramenta amplamente utilizada no desenvolvimento de aplicativos sem a necessidade de codificação manual. Embora não utilizemos o GeneXus neste curso, é importante conhecer sua existência e o impacto das ferramentas CASE no desenvolvimento ágil de software.

Assim, IDEs e ferramentas CASE desempenham um papel vital na modernização do desenvolvimento de software, tornando o processo mais ágil e acessível, tanto para desenvolvedores experientes quanto para aqueles que estão iniciando na área.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Modelagem de Software é Difícil? | “Ver” vs “Enxergar””, que está disponível no canal Fabio Akita no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamentos da programação de computadores**. 3.ed. Pearson: 2012.

BRAGA, P. H. **Teste de software**. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). **Arquitetura de software**. Pearson: 2017.

GALLOTTI, G. M. A. **Qualidade de software**. Pearson: 2015.

MEDEIROS, E. **Desenvolvendo software com UML 2.0 definitivo**. Pearson: 2004.

MORAIS, I. S. de. (Org.). **Engenharia de software**. Pearson: 2017.

PFLEEGER, S. L. **Engenharia de software: teoria e prática**. 2.ed. Pearson: 2003.

SOMMERVILLE, I. **Engenharia de software**. 10.ed. Pearson: 2019.